

Widely used Activation function inside the neurons

Moin Mostakim

Department of Computer Science and Engineering
Faculty of School of Data Science

October 2023

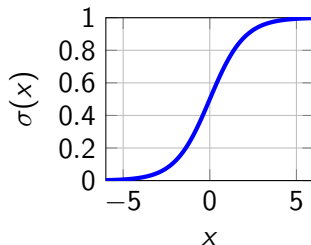


Contents of the slide

- 1 Sigmoid Activation Function
- 2 Hyperbolic Tangent (Tanh) Activation Function
- 3 Rectified Linear Unit (ReLU) Activation Function
- 4 Leaky Rectified Linear Unit (Leaky ReLU) Activation Function
- 5 Exponential Linear Unit (ELU) Activation Function
- 6 Swish Activation Function
- 7 Gated Linear Unit (GLU) Activation Function
- 8 Softmax Activation Function

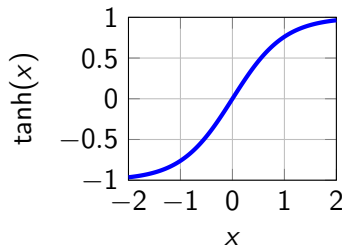
Sigmoid Activation Function

- Formula: $\sigma(x) = \frac{1}{1+e^{-x}}$
- Range: $(0, 1)$
- First-order Derivative:
 $\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$
- Output:
 - Shape: S-shaped curve.
 - Use Cases: Binary classification, sigmoid neurons in the output layer.
 - Benefits: Smooth gradient, suitable for converting network outputs to probabilities.



Hyperbolic Tangent (Tanh) Activation Function

- Formula: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- Range: $(-1, 1)$
- First-order Derivative:
 $\tanh'(x) = 1 - \tanh^2(x)$
- Output:
 - Shape: S-shaped curve similar to sigmoid.
 - Use Cases: Regression, classification.
 - Benefits: Centered around zero, mitigates vanishing gradient problem, and provides smooth gradients.



Rectified Linear Unit (ReLU) Activation Function

- Formula: $\text{ReLU}(x) = \max(0, x)$
- Range: $[0, \infty)$
- First-order Derivative:

$$\text{ReLU}'(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

- Output:
 - Shape: Linear for positive values, zero for negatives.
 - Use Cases: Hidden layers in most neural networks.
 - Benefits: Efficient, mitigates vanishing gradient, induces sparsity.

Leaky Rectified Linear Unit (Leaky ReLU) Activation Function

- Formula: $\text{LeakyReLU}(x, \alpha) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{if } x < 0 \end{cases}$ *Range* : $(-\infty, \infty)$

- First-order Derivative:

$$\text{LeakyReLU}'(x, \alpha) = \begin{cases} 1 & \text{if } x \geq 0 \\ \alpha & \text{if } x < 0 \end{cases}$$

- Output:
 - Shape: Linear for positive values, non-zero slope for negatives.
 - Use Cases: Alternative to ReLU to prevent "dying ReLU" problem.
 - Benefits: Addresses "dying ReLU" issue, retains sparsity.

Exponential Linear Unit (ELU) Activation Function

- Formula:
$$\text{ELU}(x, \alpha) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha(e^x - 1) & \text{if } x < 0 \end{cases} \text{Range : } (-\infty, \infty)$$

- First-order Derivative:

$$\text{ELU}'(x, \alpha) = \begin{cases} 1 & \text{if } x \geq 0 \\ \alpha e^x & \text{if } x < 0 \end{cases}$$

- Output:

- Shape: Smooth S-shaped curve with an exponential increase for negative values.
- Use Cases: An alternative to ReLU with smoother gradients.
- Benefits: Smoother gradients, better training on negative values.

Swish Activation Function

- Formula: $\text{Swish}(x) = x \cdot \sigma(x)$
- Range: $(-\infty, \infty)$
- First-order Derivative: $\text{Swish}'(x) = \text{Swish}(x) + \sigma(x) \cdot (1 - \text{Swish}(x))$
- Output:
 - Shape: Smooth, non-monotonic curve.
 - Use Cases: Considered in some architectures as an alternative to ReLU.
 - Benefits: Smoothness, performance improvements observed in deep networks.

Gated Linear Unit (GLU) Activation Function

- Formula: $\text{GLU}(x) = x \cdot \sigma(g(x))$
- Range: $(-\infty, \infty)$
- First-order Derivative:
$$\text{GLU}'(x) = \sigma(g(x)) + x \cdot \sigma'(g(x)) \cdot (1 - \sigma(g(x)))$$
- Output:
 - Shape: Complex, involving a sigmoid gate.
 - Use Cases: Used in architectures like the Transformer and other sequence-to-sequence models.
 - Benefits: Enables modeling dependencies in sequences, better than standard RNNs.

Softmax Activation Function

- Formula (for class i): $\text{Softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$
- Range: (0, 1)
- First-order Derivative:
$$\frac{\partial}{\partial x_i} \text{Softmax}(x)_j = \text{Softmax}(x)_i \cdot (\delta_{ij} - \text{Softmax}(x)_j)$$
- Output:
 - Shape: Probability distribution over classes.
 - Use Cases: Used in the output layer of multi-class classification for probability distribution over classes.
 - Benefits: Converts scores to class probabilities, essential for classification tasks.